

Practical: Binary Classification using Deep Neural Network (IMDB Movie Review Sentiment Analysis)

Laboratory Practice V – Computer Engineering

1 Aim

Classify movie reviews into “positive” reviews and “negative” reviews, just based on the text content of the reviews, using Deep Neural Network. Use the IMDB dataset.

1.1 Objective

Students should be able to classify movie reviews into positive reviews and negative reviews on the IMDB Dataset using a Deep Neural Network.

1.2 Outcome

Student can apply the technique of Deep Neural Network for implementing binary classification.

2 Theory

2.1 What is Classification?

Classification is a type of **supervised learning** in machine learning that involves categorizing data into predefined classes or categories based on a set of features. A model is trained on a labeled dataset, where each data point has a known class label. The model learns to associate the input features with the corresponding class labels and can then be used to classify new, unseen data.

Examples:

- Classify email as spam or not-spam
- Predict if a patient has a disease or not
- Classify movie reviews as positive or negative

2.2 What is Binary Classification?

Binary classification is a special case of classification where there are only **two possible classes**:

- Positive (1) or Negative (0)
- Spam or Not-Spam
- Yes or No

In this practical, the two classes are: **Positive Review (1)** and **Negative Review (0)**.

2.3 How Deep Neural Network Works for Classification

The basic architecture of a DNN for classification consists of three main parts:

1. **Input Layer** — receives the preprocessed input data (TF-IDF features)
2. **Hidden Layers** — neurons apply linear transformations and nonlinear activations (ReLU) to extract relevant patterns
3. **Output Layer** — produces the predicted class (using sigmoid for binary)

During training, the DNN adjusts its weights and biases to minimize the **Binary Cross-Entropy loss** using the **Adam optimizer**.

Key Advantage: Deep neural networks can learn **hierarchical representations** — each layer captures more complex and abstract features than the previous one.

2.4 IMDB Dataset

Property	Details
Total Reviews	50,000 (approx 25,000 positive + 25,000 negative)
Format	Each review is raw text with a sentiment label
Labels	“positive” (1) or “negative” (0)
Source	IMDB website
Challenge	Reviews contain subjective language, sarcasm, complex sentences

2.5 Key Concepts Used

Concept	Meaning
TF-IDF	Term Frequency–Inverse Document Frequency. Converts text to numerical features based on word importance
Binary Cross-Entropy	Loss function for binary classification. Measures how far the predicted probability is from the actual label
Sigmoid Activation	Squishes output between 0 and 1 (probability of being positive)
ReLU Activation	$f(x) = \max(0, x)$ — used in hidden layers
Adam Optimizer	Adaptive optimizer that adjusts learning rate for each parameter
Validation Split	Portion of data used to check model performance during training
Epoch	One complete pass through the entire training dataset

3 Code

Listing 1: IMDB Movie Review Binary Classification using DNN

```

1 # =====
2 # 1. IMPORT LIBRARIES
3 # =====
4 import numpy as np

```

```
5 import pandas as pd
6 import matplotlib.pyplot as plt
7 import re
8
9 from sklearn.model_selection import train_test_split
10 from sklearn.feature_extraction.text import TfidfVectorizer
11
12 from tensorflow.keras.models import Sequential
13 from tensorflow.keras.layers import Dense
14
15 # =====
16 # 2. LOAD DATASET
17 # =====
18 df = pd.read_csv(
19     "IMDB_Dataset.csv",
20     encoding_errors='ignore',
21     on_bad_lines='skip',
22     engine='python'
23 )
24 print("Dataset Loaded")
25 print(df.head())
26 print(df.columns)
27
28 # =====
29 # 3. CHECK & CLEAN DATA
30 # =====
31 df = df.dropna()
32 print(df['sentiment'].unique())
33 print(df.isnull().sum())
34
35 # =====
36 # 4. FIX COLUMN NAMES
37 # =====
38 df.columns = ['review', 'sentiment']
39
40 # =====
41 # 5. CLEAN TEXT
42 # =====
43 def clean_text(text):
44     text = str(text)
45     text = text.lower()
46     text = re.sub(r'<.*?>', '', text) # remove HTML tags
47     text = re.sub(r'[^a-zA-Z ]', '', text) # remove symbols
48     return text
49
50 df['review'] = df['review'].apply(clean_text)
51
52 # =====
53 # 6. CONVERT LABELS
54 # =====
55 df['sentiment'] = df['sentiment'].map({'positive': 1, 'negative': 0})
56 print("\nSentiment Count:")
57 print(df['sentiment'].value_counts())
```

```

58
59 # =====
60 # 7. TF-IDF VECTORIZATION
61 # =====
62 vectorizer = TfidfVectorizer(max_features=5000)
63 X = vectorizer.fit_transform(df['review']).toarray()
64 y = df['sentiment']
65
66 # =====
67 # 8. TRAIN TEST SPLIT
68 # =====
69 X_train, X_test, y_train, y_test = train_test_split(
70     X, y, test_size=0.2, random_state=42
71 )
72
73 # =====
74 # 9. BUILD DNN MODEL
75 # =====
76 model = Sequential()
77 model.add(Dense(128, activation='relu', input_shape=(X_train.shape
78     [1],)))
79 model.add(Dense(64, activation='relu'))
80 model.add(Dense(1, activation='sigmoid'))
81
82 model.compile(
83     optimizer='adam',
84     loss='binary_crossentropy',
85     metrics=['accuracy']
86 )
87 model.summary()
88
89 # =====
90 # 10. TRAIN MODEL
91 # =====
92 history = model.fit(
93     X_train, y_train,
94     epochs=10,
95     batch_size=32,
96     validation_data=(X_test, y_test)
97 )
98
99 # =====
100 # 11. PLOT ACCURACY GRAPH
101 # =====
102 plt.plot(history.history['accuracy'], label='Training Accuracy')
103 plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
104 plt.title("Accuracy Graph")
105 plt.xlabel("Epochs")
106 plt.ylabel("Accuracy")
107 plt.legend()
108 plt.show()
109
110 # =====

```

```

110 # 12. FINAL ACCURACY
111 # =====
112 loss, accuracy = model.evaluate(X_test, y_test)
113 print("\nFinal Accuracy:", accuracy)

```

4 Output

4.1 Dataset Info

```

Dataset Loaded
Columns: ['review', 'sentiment']
Unique sentiments: ['positive', 'negative']
Sentiment Count:
    negative (0): 24813
    positive (1): 24799

```

4.2 Model Summary

```

Model: "sequential"
+-----+-----+-----+
| Layer (type)          | Output Shape      | Param #   |
+-----+-----+-----+
| dense (Dense)         | (None, 128)       | 640,128   |
| dense_1 (Dense)       | (None, 64)        | 8,256     |
| dense_2 (Dense)       | (None, 1)         | 65        |
+-----+-----+-----+
Total params: 648,449   (2.47 MB)

```

4.3 Training Output (Selected Epochs)

```

Epoch 1/10 - accuracy: 0.8659 - loss: 0.3120
              val_accuracy: 0.8849 - val_loss: 0.2655
Epoch 3/10 - accuracy: 0.9286 - loss: 0.1797
              val_accuracy: 0.8835 - val_loss: 0.2979
Epoch 5/10 - accuracy: 0.9933 - loss: 0.0223
              val_accuracy: 0.8771 - val_loss: 0.6191
Epoch 10/10 - accuracy: 1.0000 - loss: 0.0000
              val_accuracy: 0.8792 - val_loss: 1.1256

```

Note: Training accuracy reaches 100% but validation accuracy stays around 87.9% — this indicates **overfitting**. The model memorized the training data but doesn't generalize well to new data. Solutions: add Dropout, reduce epochs, or use more data.

4.4 Final Test Accuracy

```

Final Accuracy: 0.8792 (87.92%)

```

5 Code Explanation (Hinglish)

5.1 Is Practical Ka Aim Kya Hai?

Humein IMDB ki **50,000 movie reviews** dekhke batana hai ki review **positive** hai ya **negative**. Sirf text content se — koi rating ya star nahi milega, bas review padhke classify karna hai.

Yeh ek **Binary Classification** problem hai — sirf 2 categories: Positive (1) ya Negative (0).

5.2 Karna Kya Hai? (Har Cell Ka Matlab)

5.2.1 Cell 1: Import Libraries

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt, re
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

- numpy/pandas — data handle karne ke liye
- matplotlib — accuracy graph banana ke liye
- re — regular expressions se text saaf karne ke liye
- TfidfVectorizer — text ko numbers mein convert karne ke liye
- Sequential/Dense — neural network banana ke liye

Simple: Sab tools import kiye.

5.2.2 Cell 2: Load Dataset

```
df = pd.read_csv("IMDB_Dataset.csv", encoding_errors='ignore',
                 on_bad_lines='skip', engine='python')
```

IMDB CSV file load ki jo 50,000 reviews hai — har row mein ek “review” (text) aur ek “sentiment” (positive/negative) hai.

Simple: Excel jaisi file se reviews aur unke labels load kiye.

5.2.3 Cell 3–4: Check & Clean Data

```
df = df.dropna() # NaN values hatao
df.columns = ['review', 'sentiment'] # column names fix karo
```

Missing values hata diye aur column names fix kiye. Verify kiya ki data mein sirf “positive” aur “negative” labels hain.

Simple: Data saaf kiya — missing entries hatai.

5.2.4 Cell 5: Clean Text

```
def clean_text(text):
    text = str(text)
    text = text.lower()                # lowercase
    text = re.sub(r'<.*?>', '', text)  # HTML hatao
    text = re.sub(r'[^a-zA-Z ]', '', text) # symbols hatao
    return text
df['review'] = df['review'].apply(clean_text)
```

Har review mein:

- Sab lowercase mein convert kiya
- HTML tags (jaise
) hataye
- Numbers aur symbols (!, ?, @, etc.) hataye
- Sirf alphabets aur spaces rakhe

Kyun? Machine ko text samajhne ke liye pehle text saaf karna padta hai — extra noise hata ke sirf meaningful words rakhte hain.

Simple: Reviews se gandagi (HTML, symbols) saaf ki.

5.2.5 Cell 6: Convert Labels to Numbers

```
df['sentiment'] = df['sentiment'].map({'positive': 1, 'negative': 0})
```

“positive” ko 1 aur “negative” ko 0 mein convert kiya. Machine numbers samajhti hai, words nahi.

Output: ~24,813 negative aur ~24,799 positive — almost balanced dataset!

Simple: Labels ko numbers mein badla.

5.2.6 Cell 7: TF-IDF Vectorization (Text → Numbers)

```
vectorizer = TfidfVectorizer(max_features=5000)
X = vectorizer.fit_transform(df['review']).toarray()
y = df['sentiment']
```

Yeh sabse important step hai!

- **TF-IDF** = Term Frequency × Inverse Document Frequency
- Har review ko **5000 numbers ki array** mein convert karta hai
- Jo words zyada important hain (rare but meaningful) unko zyada weight milta hai
- Jo words har jagah hain (“the”, “is”, “a”) unko kam weight milta hai
- X = 50,000 rows × 5,000 columns ki matrix ban gayi
- y = labels (0 ya 1)

Simple: Text ko numbers mein badla taaki machine samajh sake. Important words ko zyada weightage diya.

5.2.7 Cell 8: Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Data ko 2 parts mein toda:

- **80% Training** — model isse seekhega (~40,000 reviews)
- **20% Testing** — model ki ability test karenge (~10,000 reviews)

Simple: Practice questions (train) aur actual exam (test) alag kiye.

5.2.8 Cell 9: Build DNN Model

```
model = Sequential()
model.add(Dense(128, activation='relu', input_shape=(5000,)))
model.add(Dense(64, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(optimizer='adam', loss='binary_crossentropy',
               metrics=['accuracy'])
```

Layer by layer samjho:

1. **Dense(128, relu)** — 5000 input features lete hai, 128 neurons se process karte hain (6,40,128 parameters!)
2. **Dense(64, relu)** — aur complex patterns nikalte hain
3. **Dense(1, sigmoid)** — **1 neuron** jo 0 se 1 ke beech probability deta hai
 - Output > 0.5 → Positive review
 - Output ≤ 0.5 → Negative review

Compile settings:

- **adam** — smart optimizer jo weights update karega
- **binary_crossentropy** — binary classification ke liye loss function
- **accuracy** — kitne % sahi predict kiye track karega

Total parameters: 6,48,449 — yeh sab weights hain jo model seekhega.

Simple: 3-layer ka neural network banaya: 5000 words → 128 neurons → 64 neurons → 1 output (positive ya negative).

5.2.9 Cell 10: Train Model

```
history = model.fit(X_train, y_train, epochs=10,
                    batch_size=32, validation_data=(X_test, y_test))
```

- **epochs=10** — poora dataset 10 baar dekhega
- **batch_size=32** — ek baar mein 32 reviews process karega

- `validation_data` — har epoch ke baad test data pe check karega
- Training accuracy: 86% (Epoch 1) → 100% (Epoch 10)
- Validation accuracy: 88.5% (Epoch 1) → 87.9% (Epoch 10)

Observation: Training 100% pe pahunch gayi lekin validation 87.9% pe ruki — yeh **overfitting** hai! Model ne training data ratt liya lekin naye data pe utna accha nahi kar raha.

Simple: Model ne 10 baar poora data dekhke seekha. Training pe 100% accuracy mili lekin test pe 88% — thoda overfitting hua.

5.2.10 Cell 11: Plot Accuracy Graph

```
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.show()
```

Graph mein training accuracy upar jaati hai (100% tak) lekin validation accuracy ek level pe ruk jaati hai — classic overfitting pattern.

Simple: Graph se dekhte hain ki model seekh raha hai ya ratt raha hai.

5.2.11 Cell 12: Final Test Accuracy

```
loss, accuracy = model.evaluate(X_test, y_test)
print("Final Accuracy:", accuracy) # 87.92%
```

Final test accuracy: **87.92%** — matlab 10 mein se ~9 reviews sahi classify kiye!

Simple: Final exam diya — 87.92% marks aaye.

5.3 Final Outcome Kya Milta Hai?

1. **Trained DNN Model** — jo movie review text dekhke positive/negative bata sakta hai
2. **87.92% Test Accuracy** — 10 mein se ~9 reviews sahi classify
3. **Accuracy Graph** — training vs validation comparison
4. **Overfitting awareness** — model ne training data ratt liya (100%) lekin test pe 88%

Bottom Line: Is practical mein humne dekha ki kaise raw text data (movie reviews) ko TF-IDF se numbers mein convert karke ek Deep Neural Network (3 Dense layers) se binary classification ki. Model ne 87.92% accuracy achieve ki.

6 How to Run

6.1 On Google Colab

1. Go to `colab.research.google.com`
2. Upload the `.ipynb` notebook
3. Upload `IMDB_Dataset.csv` to Colab
4. Run all cells (Runtime → Run all)

6.2 On Fresh Ubuntu PC

1. Update system:

```
sudo apt update && sudo apt upgrade -y
```

2. Install Python 3 and pip:

```
sudo apt install python3 python3-pip python3-venv -y
```

3. Create virtual environment:

```
python3 -m venv dl_env  
source dl_env/bin/activate
```

4. Install libraries:

```
pip install numpy pandas matplotlib scikit-learn tensorflow  
jupyter
```

5. Copy files and run:

```
mkdir ~/IMDB_Practical && cd ~/IMDB_Practical  
# Copy IMDB_Dataset.csv and .ipynb here  
jupyter notebook  
# Open notebook and run all cells
```

Troubleshooting:

- If pip install tensorflow fails, try: pip install tensorflow-cpu
- Make sure IMDB_Dataset.csv is in the **same folder** as the notebook

7 FAQ

1. What is Binary Classification?

A classification task with only 2 possible output classes (e.g., positive/negative, yes/no, 0/1).

2. What is Binary Cross-Entropy?

A loss function used for binary classification. It measures the difference between the predicted probability and the actual label. Formula: $L = -[y \log(p) + (1 - y) \log(1 - p)]$

3. What is Validation Split?

A portion of data (here 20%) kept aside to evaluate the model during training. It helps detect overfitting.

4. What is an Epoch?

One complete pass through the entire training dataset. If epochs=10, the model sees all training data 10 times.

5. What is Adam Optimizer?

An adaptive learning rate optimization algorithm. It combines the benefits of AdaGrad and RMSProp, automatically adjusting the learning rate for each parameter.

8 Conclusion

In this practical, we implemented Binary Classification of IMDB movie reviews using a Deep Neural Network. The text data was preprocessed by removing HTML tags, symbols, and converting to lowercase. TF-IDF vectorization was used to convert text into numerical features (5000 features per review). A 3-layer DNN with Dense(128, ReLU) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dense(1, Sigmoid) architecture was built and trained using Adam optimizer with Binary Cross-Entropy loss for 10 epochs. The model achieved a final test accuracy of approximately 87.92%, demonstrating the effectiveness of Deep Neural Networks for text-based binary classification tasks.